Семинар 11. Распределенные файловые системы: NFS, GFS, HDFS, IPFS

Распределенные вычисления

Георгий Семенов

Институт прикладных компьютерных наук
Университет ИТМО

весна 2026

Организационные моменты

- Через неделю – (7 мая, 23:59) дедлайн по домашнему заданию

Содержание

1. «Обычные» файловые системы: FAT, NTFS, ext4
2. Распределенные файловые системы
3. Централизованные DFS: NFS, AFS
4. Параллельные DFS: GFS, HDFS
5. Peer-to-peer DFS: IPFS
6. Полезные техники вокруг данных

Понятие файловой системы

Файловая система

Это программное обеспечение, которое управляет тем, как данные хранятся и извлекаются на запоминающем устройстве.

Основные задачи:

- Организация данных в файлы и каталоги
- Управление пространством на диске
- Контроль доступа к файлам (права доступа)
- Отслеживание метаданных (имя, размер, время создания)
- Обеспечение надежности данных

FAT (File Allocation Table)

История: разработана компанией Microsoft в 1977 году для дискет, позже использовалась для жёстких дисков

Структура:

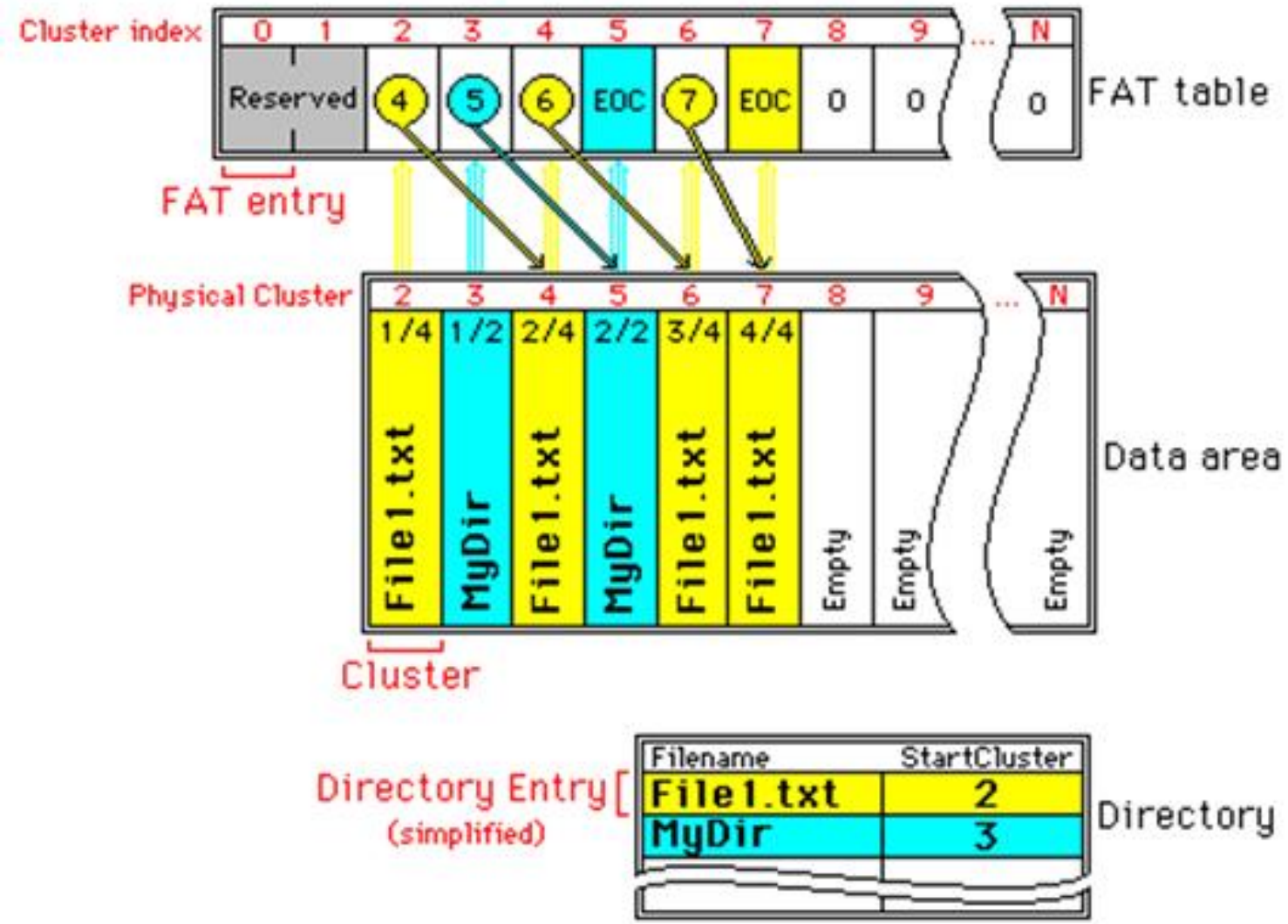
- **Boot sector** — загрузочный сектор с информацией о ФС
- **File Allocation Table** — таблица распределения (несколько копий)
- **Root directory** — корневой каталог
- **Data area** — область данных

Варианты: FAT12 (дискеты), FAT16, FAT32

Недостатки:

- Нет встроенной защиты доступа
- Фрагментация диска
- Ограничения на размер файлов (FAT32: 4 GB)

Структура таблицы



Подробнее про FAT

NTFS (NT File System)

История: разработана Microsoft для Windows NT (1993), стандарт для Windows

Ключевые возможности:

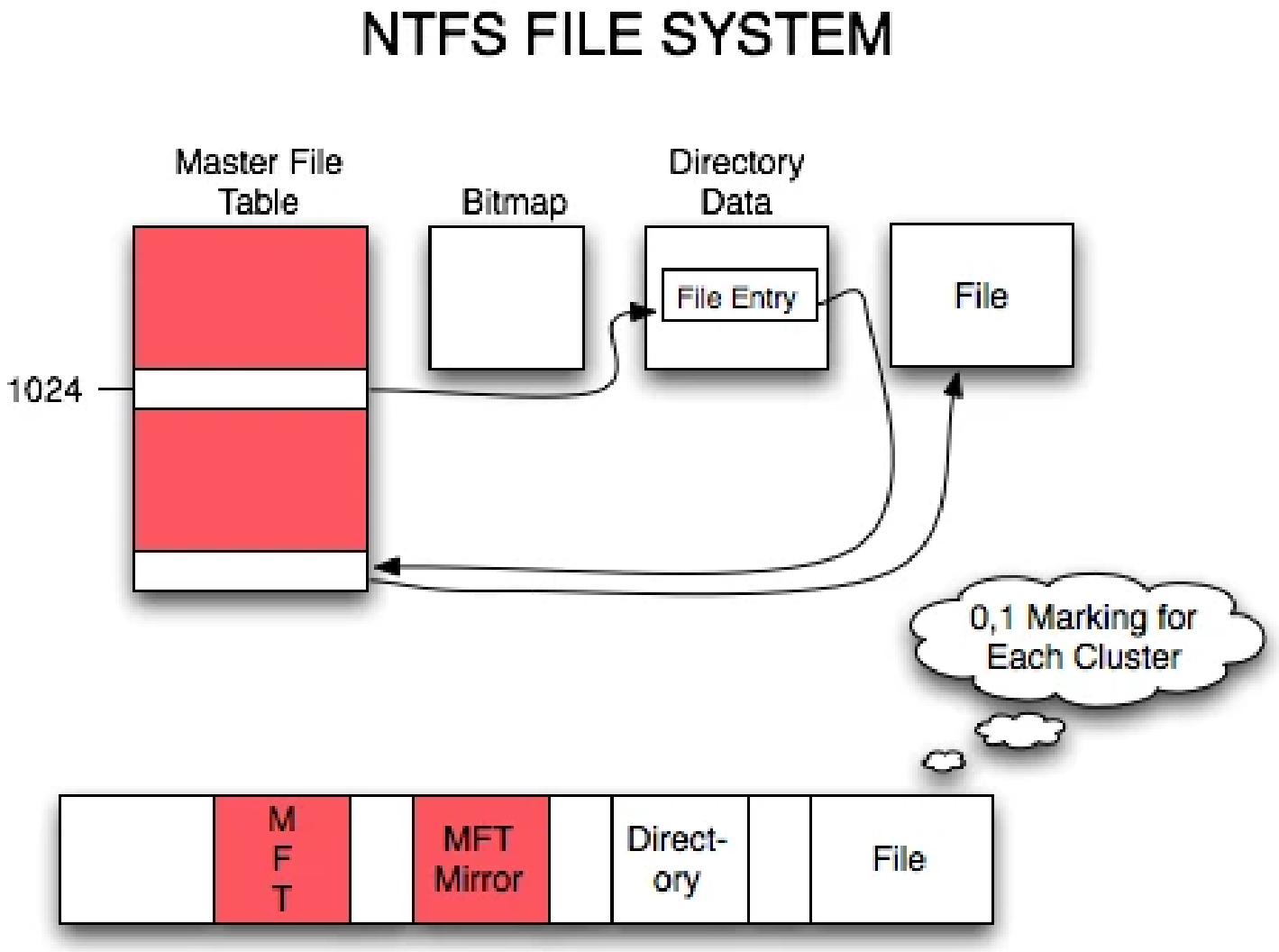
- Журналирование (journaling) для надежности
- Файлы больше 4 GB и диски больше 2 TB
- Контроль доступа (ACL — Access Control Lists)
- Шифрование файлов (EFS)
- Жесткие и символические ссылки

Структура:

- **MFT** (Master File Table) — основная таблица файлов
- Каждый файл — это набор атрибутов в MFT

Сравнение с FAT: FAT проще, но NTFS надежнее и функциональнее

Структура таблицы



Файловые системы в Linux: ext4

История линейки ext: ext (1992) → ext2 (1993) → ext3 (2001) → ext4 (2008)

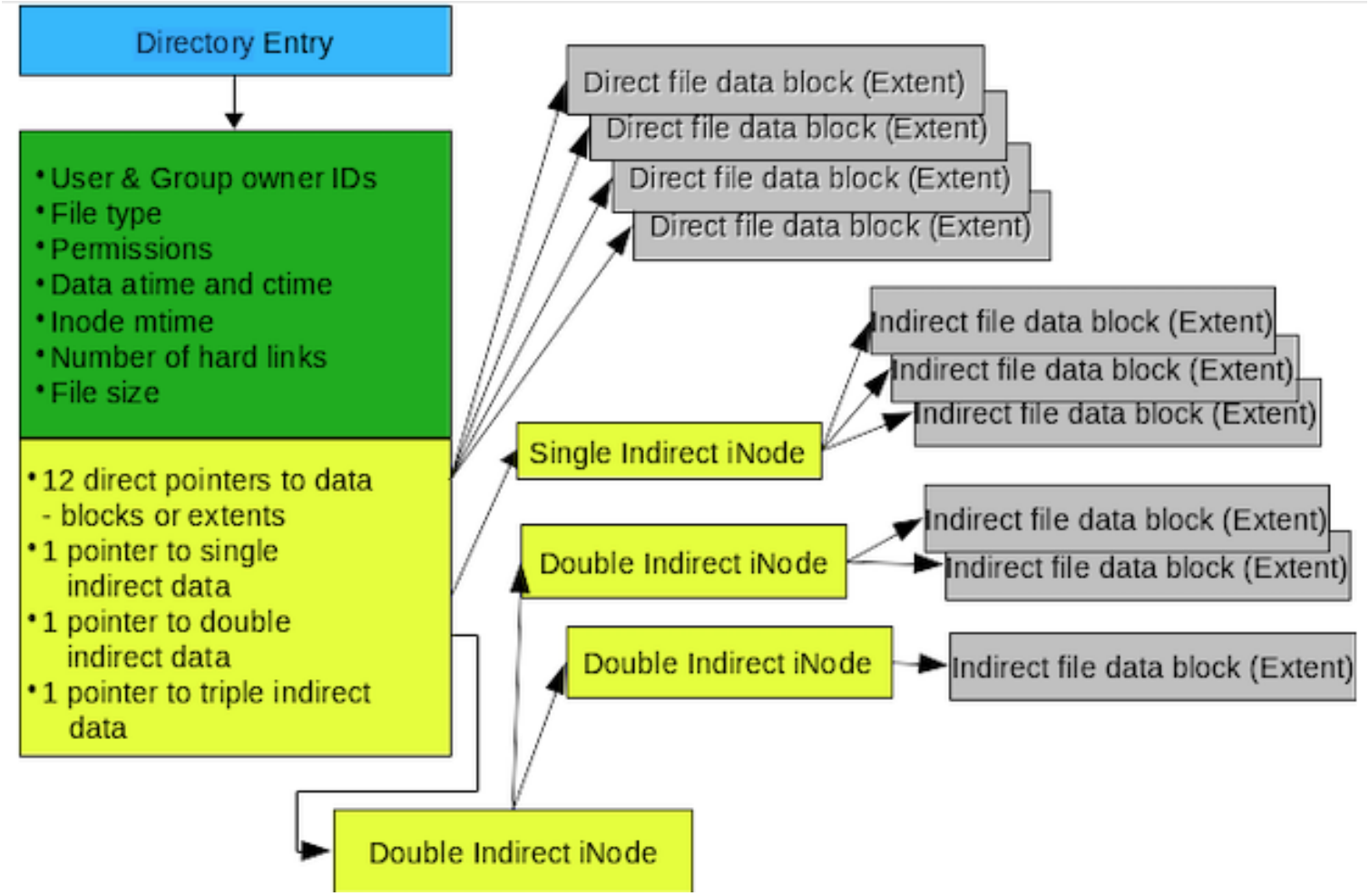
ext4 особенности:

- Журналирование (как NTFS)
- Extents вместо блок-карт (улучшена фрагментация)
- Спарс-файлы (sparse files)
- Временные метки с наносекундной точностью
- Размер файла до 16 TB, раздела до 1 EB

Структура на диске:

- **Superblock** — информация о ФС
- **Block Groups** — группы блоков с битовыми картами
- **Inode table** — таблица инодов (метаданные файлов)
- **Data blocks** — блоки данных

Структура ext4



Содержание

1. «Обычные» файловые системы: FAT, NTFS, ext4
2. Распределенные файловые системы
3. Централизованные DFS: NFS, AFS
4. Параллельные DFS: GFS, HDFS
5. Peer-to-peer DFS: IPFS
6. Полезные техники вокруг данных

Понятие распределенной файловой системы

DFS

Файловая система, которая хранит файлы на нескольких серверах или компьютерах, объединяя их в единое виртуальное пространство имён.

Преимущества:

- **Масштабируемость** — добавляем новые серверы для увеличения хранилища
- **Отказоустойчивость** — потеря одного сервера не потеряет всё
- **Распределённый доступ** — несколько клиентов работают параллельно
- **Производительность** — балансировка нагрузки между серверами

Примеры: NFS, AFS, GFS, HDFS, Ceph, Lustre, IPFS

Отличия от обычных файловых систем

Обычная ФС (локальная):

- Одна машина
- Простая архитектура
- Быстрый доступ
- Ограничены ёмкостью диска

Распределённая ФС:

- Несколько машин
- Сложная архитектура
- Задержки сети
- Масштабируются горизонтально

Дополнительные вызовы DFS:

- **Консистентность** — синхронизация данных между узлами
- **Отказы** — потеря связи, сбой сервера
- **Производительность** — задержки сети усложняют оптимизацию
- **Безопасность** — защита передачи данных по сети

Типы распределённых файловых систем

1. **Централизованные DFS**
   - Центральный сервер управляет доступом и распределением
   - Примеры: NFS, AFS
2. **Распределённые (параллельные) DFS**
   - Серверы метаданных управляют пространством имён
   - Серверы хранилища отвечают за данные
   - Примеры: GFS, HDFS, Lustre, Ceph
3. **P2P DFS**
   - Каждый узел одновременно клиент и сервер
   - Нет центрального управления
   - Примеры: IPFS, BitTorrent, DHT

Amazon S3: отличия от DFS

aws

Get started Service guides Developer tools

Amazon Simple Storage Service
API Reference

Welcome
- S3 API Reference
  - Actions
    - Amazon S3
      - AbortMultipartUpload
      - CompleteMultipartUpload
      - CopyObject
      - CreateBucket
      - CreateBucketMetadataConfiguration
      - CreateBucketMetadataTableConfiguration
      - CreateMultipartUpload
      - CreateSession

Documentation > Amazon Simple Storage Service (S3) > API Reference

Amazon S3

PDF Markdown Focus mode

The following actions are supported by Amazon S3:

- AbortMultipartUpload
- CompleteMultipartUpload
- CopyObject
- CreateBucket
- CreateBucketMetadataConfiguration
- CreateBucketMetadataTableConfiguration
- CreateMultipartUpload
- CreateSession
- DeleteBucket
- DeleteBucketAnalyticsConfiguration

Что такое AWS?

Содержание

1. «Обычные» файловые системы: FAT, NTFS, ext4
2. Распределенные файловые системы
3. Централизованные DFS: NFS, AFS
4. Параллельные DFS: GFS, HDFS
5. Peer-to-peer DFS: IPFS
6. Полезные техники вокруг данных

NFS (Network File System)

История: разработана Sun Microsystems в 1984 году
Основная идея:

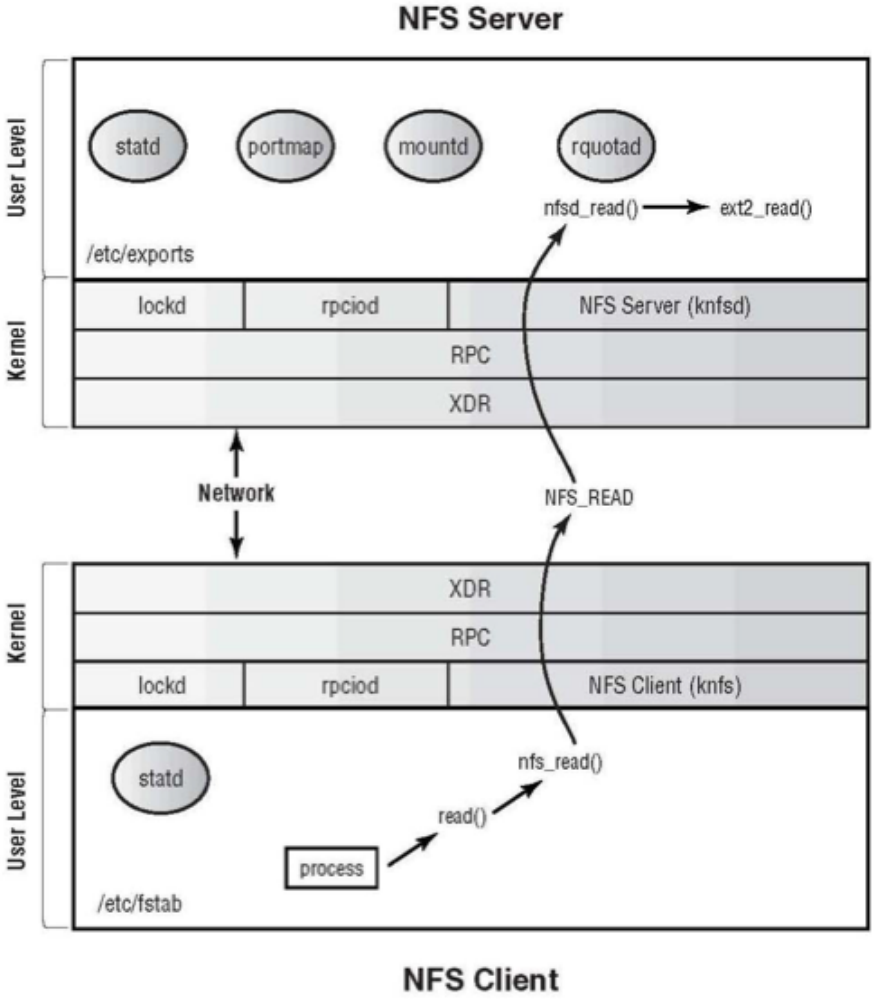
- Монтирование удалённой ФС как локальной директории
- Клиенты обращаются к серверу через RPC (Remote Procedure Call)
- Прозрачный доступ к файлам

Архитектура:

- **NFS Server** — хранит файлы, обрабатывает запросы
- **NFS Client** — кэширует данные локально
- Протокол: NFSv3 (stateless), NFSv4 (с состоянием)

Плюсы: простота, стандарт в UNIX/Linux экосистеме
Минусы: центральный сервер — узкое место, проблемы с отказоустойчивостью

NFS API



Подробнее

Сравнение с другими похожими протоколами

Network File System Comparison

Created by Dan Nanni study-notes.org	SMB	NFS	WebDAV
Primary Use	File, printer and resource sharing in mixed OS environments	File sharing in UNIX/Linux environment	Remote document access over HTTP/HTTPS
Transport Protocol	TCP (port 445)	TCP (port 2049)	HTTPS (port 443)
Authentication	NTLM, Kerberos (AD integration)	AUTH_SYS, Kerberos (RPCSEC_GSS)	OAuth, TLS certificate
Encryption	AES-128/256 GCM/CCM for SMB 3.x	Optional in NFSv4.2 (vis TLS or Kerberos)	TLS for WebDAVS
Protocol Overhead	Moderate	Low	High
Access Control	Windows ACLs and NTFS permissions	POSIX ACLs	HTTP-based auth and per-URL permissions
Compression	Yes (SMB 3.1.1)	No native support	No native support
Cross Platform	Excellent (Windows, macOS and Linux via Samba)	Linux, UNIX, limited Windows support	Excellent (all OS with WebDAV clients)

AFS (Andrew File System)

История: создана в Carnegie Mellon University (1980-е)
Особенности:

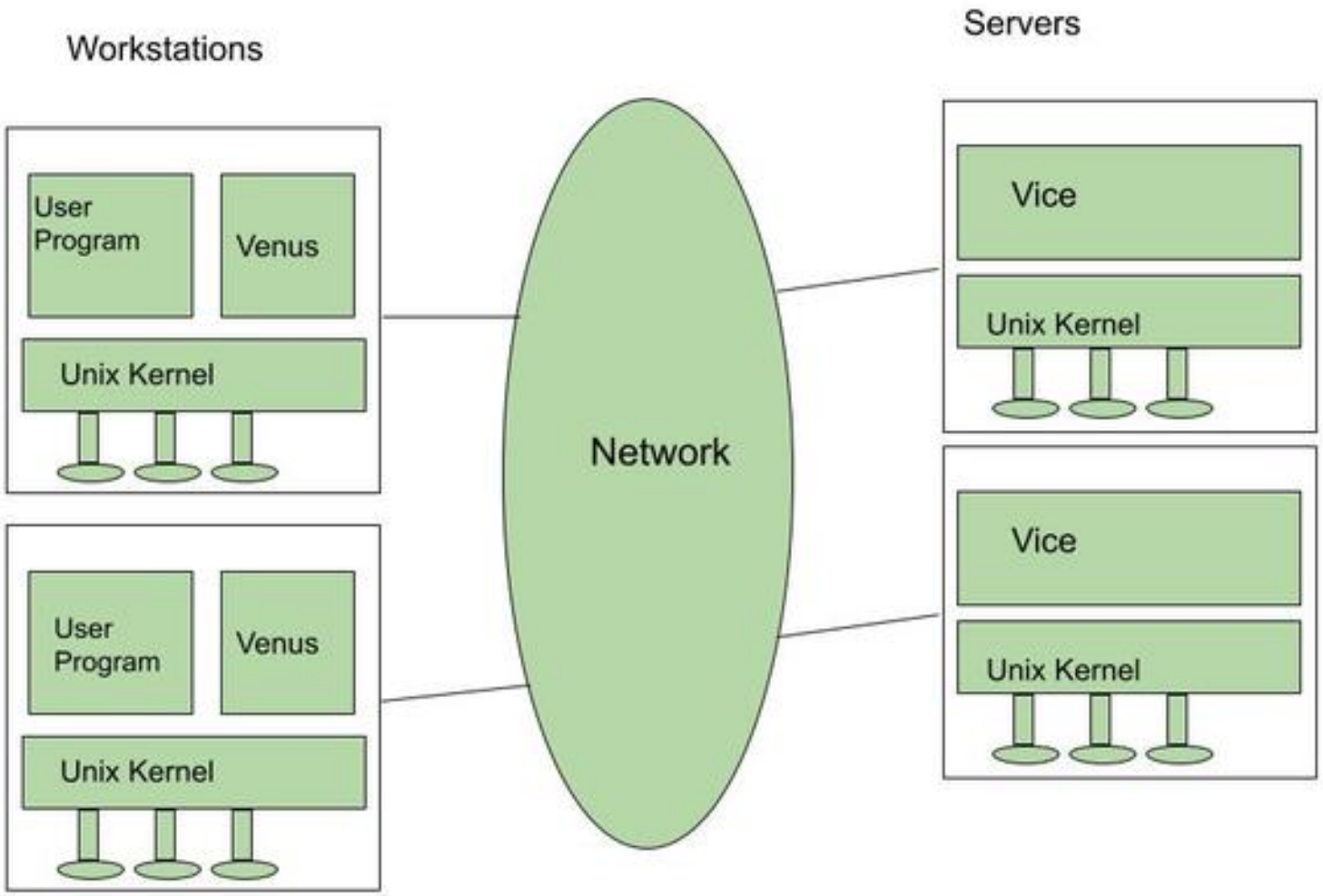
- Целое-файловое кэширование на клиентах
- Нарушение кэша (cache invalidation) через callbacks
- Иерархическое управление (ячейки — cells)
- Безопасность встроена (Kerberos)

Архитектура:

- **Vice** — серверная часть
- **Venus** — клиентская часть
- Данные обмениваются целыми файлами, не блоками

Сравнение с NFS: AFS лучше для распределённых сетей с высокой задержкой, благодаря кэшированию

AFS: картинка



Содержание

1. «Обычные» файловые системы: FAT, NTFS, ext4
2. Распределенные файловые системы
3. Централизованные DFS: NFS, AFS
4. Параллельные DFS: GFS, HDFS
5. Peer-to-peer DFS: IPFS
6. Полезные техники вокруг данных

GFS (Google File System)

Контекст: разработана Google для хранения петабайтов данных на тысячах дешёвых серверов

Архитектура:

- **Master** — управляет метаданными (одна копия, реплицирована)
- **Chunkservers** — хранят данные в chunks по 64 MB
- **Clients** — читают и пишут через master и chunkservers

Особенности:

- Высокая пропускная способность важнее низкой задержки
- Append-only семантика (добавления, не перезаписи)
- Ленивая репликация chunks
- Восстановление после сбоев на лету

Применение: MapReduce, поисковые индексы, аналитика

Подробнее

Подробнее про GFS

HDFS (Hadoop Distributed File System)

История: вдохновлена GFS, open-source реализация для Hadoop проекта

Архитектура:

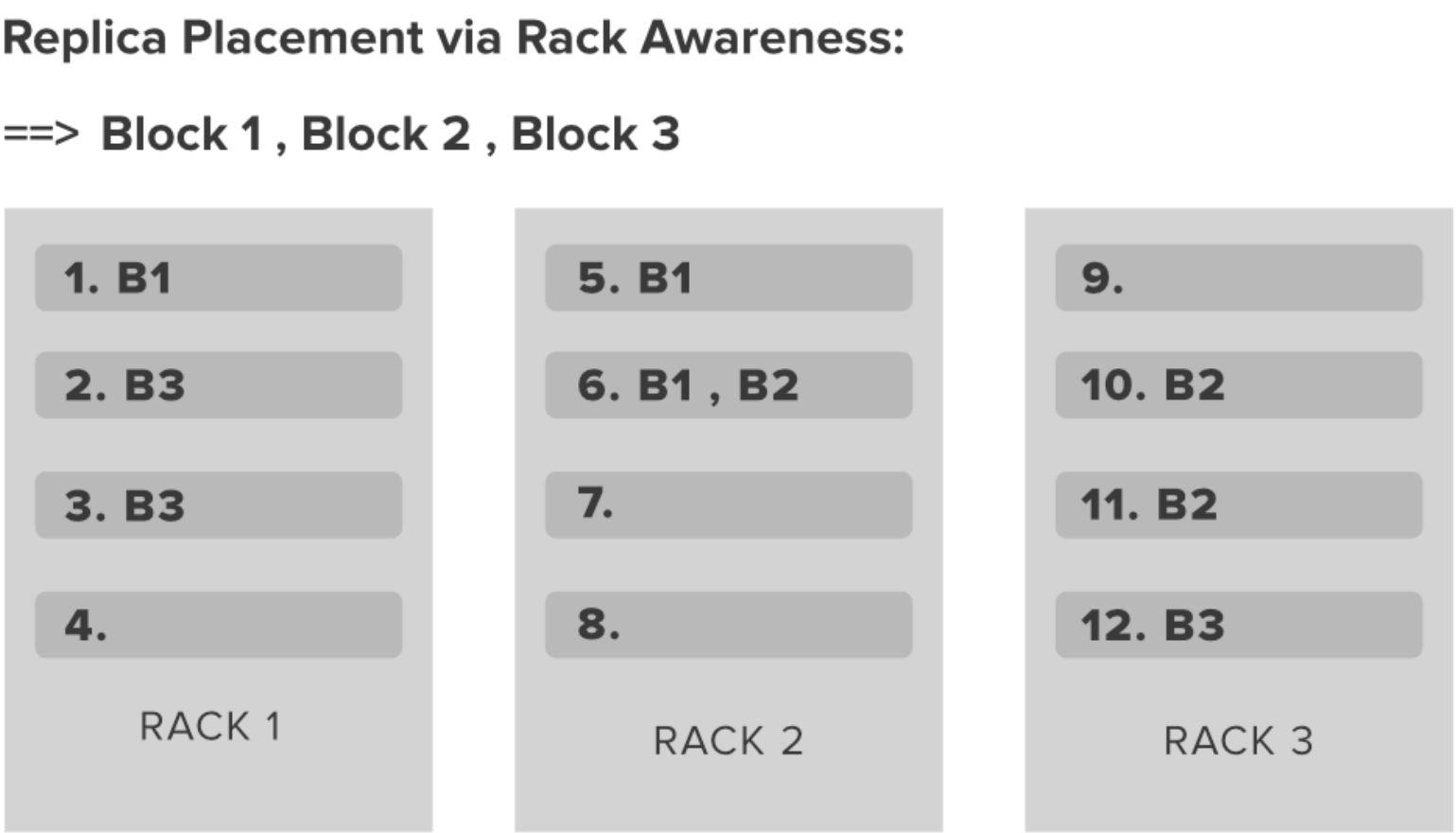
- **NameNode** — master, управляет неймспейсом
- **DataNodes** — хранят блоки (по умолчанию 128 MB или 256 MB)
- **Secondary NameNode** — помощник для контрольных точек
- Репликация блоков (по умолчанию 3 копии)

Особенности:

- Write-once модель (один раз запись, затем только чтение)
- Rack-aware placement (учитывает сетевую топологию)
- Heartbeats для мониторинга здоровья узлов

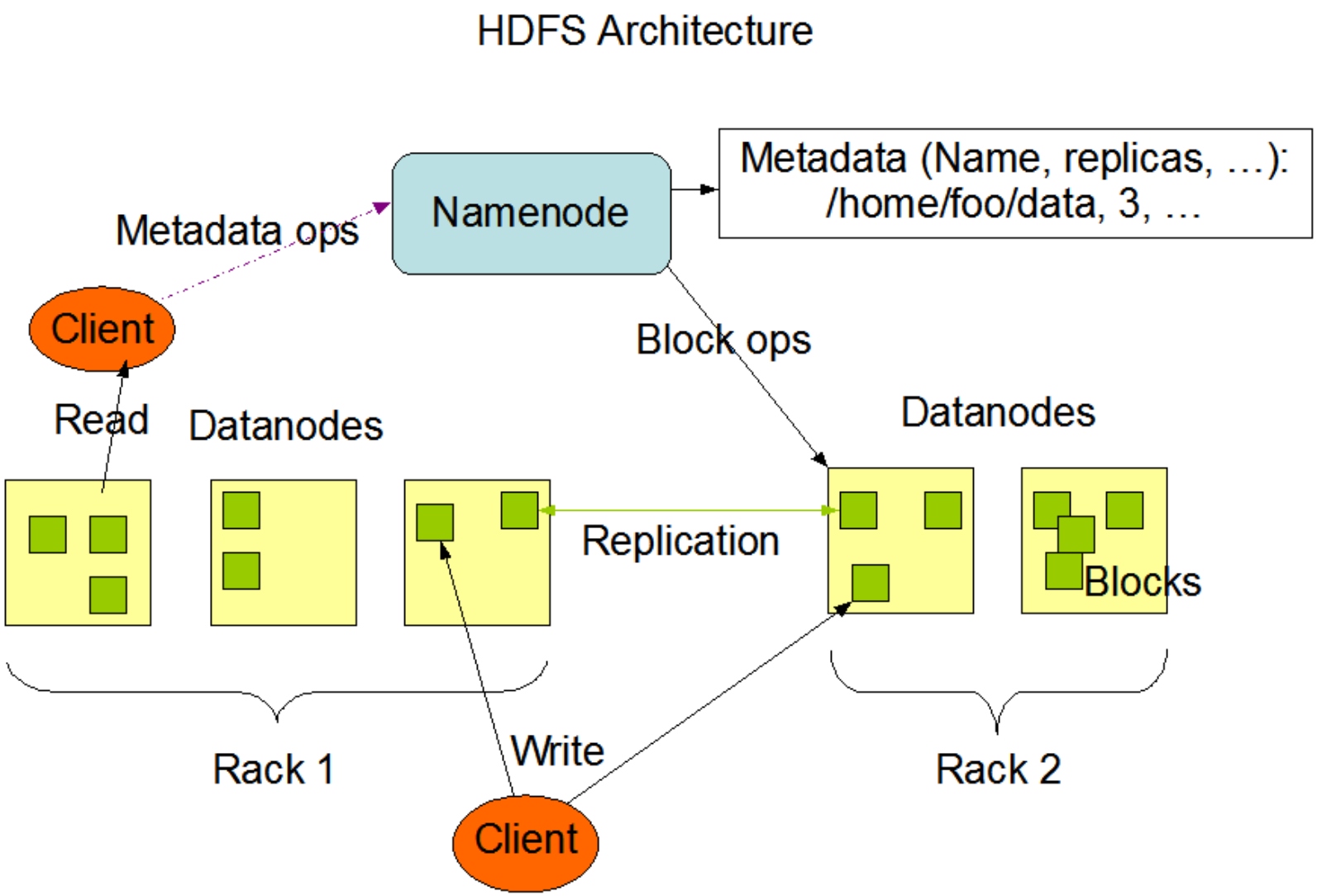
Применение: распределённые вычисления (MapReduce, Spark), Big Data аналитика

Что такое Rack Awareness?



Помогает оптимизировать размещение данных и реплик: чтобы минимизировать сетевой трафик между узлами и повысить отказоустойчивость (подробнее)

Архитектура



подробнее

Архитектура

подробнее 1
подробнее 2

Содержание

1. «Обычные» файловые системы: FAT, NTFS, ext4
2. Распределенные файловые системы
3. Централизованные DFS: NFS, AFS
4. Параллельные DFS: GFS, HDFS
5. Peer-to-peer DFS: IPFS
6. Полезные техники вокруг данных

IPFS (InterPlanetary File System)

История: создана Juan Benet (Protocol Labs), опубликована в 2015 году

Основная идея:

- Децентрализованная файловая система на основе P2P сети
- Адресация по содержимому (content-addressing), а не по локации
- Вдохновлена BitTorrent, Git, DHT

Ключевые концепции:

- **IPFS hashes** — контентные хэши (SHA-256)
- **Merkle DAG** — направленный ациклический граф (как в Git)
- **DHT** — распределённая таблица хеширования для поиска
- **Кэширование** — промежуточные узлы кэшируют популярные данные

Применение: веб-контент, научные данные, медиа-файлы, Web3

IPFS vs традиционный HTTP

HTTP:

- Client-server
- Адресация по локации
- Зависит от доступности сервера
- Единая точка отказа

IPFS:

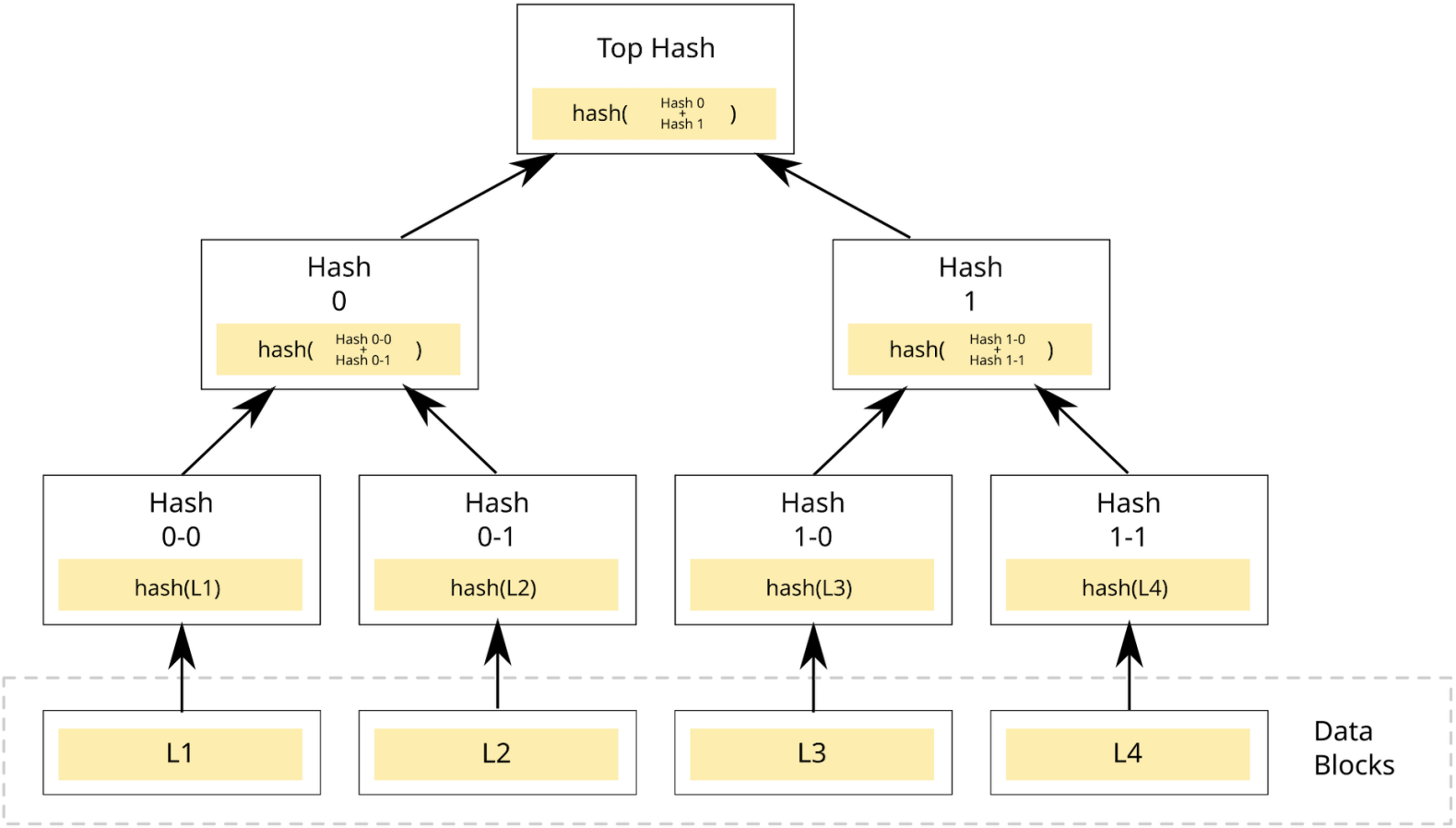
- P2P
- Адресация по контенту
- Доступна пока существует копия
- Распределённая надежность

Примеры адресов:

- HTTP: `http://example.com/image.jpg`
- IPFS: `ipfs://QmR35Vz5z...`

Хэш однозначно идентифицирует содержимое независимо от расположения.

Merkle Tree



подробнее

Содержание

1. «Обычные» файловые системы: FAT, NTFS, ext4
2. Распределенные файловые системы
3. Централизованные DFS: NFS, AFS
4. Параллельные DFS: GFS, HDFS
5. Peer-to-peer DFS: IPFS
6. Полезные техники вокруг данных

RAID

подробнее

Erasure coding

подробнее

Consistent hashing

подробнее

Итоги

- Через неделю дедлайн по домашнему заданию 4 (Raft)